COMP 8567

Advanced Systems Programming

Introduction to UNIX

Winter 2026

Dr. Prashanth Ranga

COMP 8567- Advanced Systems Programming Course Outcomes

https://ctl2.uwindsor.ca/cuma/public/courses/pdf/ee1b450a-23a6-4635-b0c6-40a47a21331f

At the end of the course, the successful student will know and be able to:

- Apply OS concepts to design algorithms to **solve systems programming problems** in a variety of different systems, such as Unix/Linux environments.
- Correctly define systems programming problems and identify and apply appropriate solutions approaches.
- Design and implement solutions that use the **hardware and/or kernel services** to solve systems programming problems involving the latest computing technologies.
- Interpret informal written descriptions of systems programming problems, and create clear, formal design specifications from them.
- Identify the contributions of open-source communities, e.g.. for Linux, and communicate the responsibility of software developers to respect licensing restrictions.
- Write reports and software documentations for problems and solutions to be used by others.
- Recognize and identify potential growth areas in operating systems' use and propose original ideas to create future applications

COMP 8567 – List of Topics

1. Introduction to UNIX
2. Advanced C programming techniques
3. Unix Input/Output
4. Process Management and Control
5. Signals
6. Inter Process Communication (Pipes)
7. Unix Shells
8. Multithreading
9. Client-Server applications
10. Systems Programming and Cloud Computing

Evaluation

- **Weekly Quizzes+ Labs (7.5%+12.5%) 10-12 Quizzes+ Labs**
- **Assignments 20 % (4 x 5%)**
- **Test 1 20 % Feb/27/2026 (Fri), 6 pm to 7:30 pm**
- **Test 2 25 % Apr/4/2026 (Sat), 10 am to 12 noon**
- **Project 10% Due Apr/13/2026, Demo/Oral Examination Apr/15-Apr/16**
- **Seminar Participation 5%**

Labs

- The system will not accept/permit submissions beyond the due date and time.

- You need to include pertinent comments explaining the working of your program

- Students are permitted to upload files only once

- The GAs and the Instructor reserve the right to confirm the originality of code submissions through <u>one or more</u> of the following methods:
  - MOSS
  - Code Walkthroughs/Explanation
  - Debugging of simple bugs introduced in the submissions (code).

Assignments

- You are required to create a 10-15 min video explaining the execution and working of your assignment solution and submit a link.
- The system will not accept/permit submissions beyond the due date and time.
- You need to include comments explaining the working of various modules of your solution.
- You are required to follow the submission instructions very carefully (reach out to the Professor if you have any questions)
- Students are permitted to upload files only once
- The GAs and the Instructor reserve the right to confirm the originality of code submissions through **one or more** of the following methods:
  - MOSS
  - Code Walkthroughs/Explanation
  - Debugging of simple bugs introduced in the submissions (code).

Test 1 and Test 2

Tests will be open book (but NO DEVICES)

No laptops, cell phones, or any other electronic equipment (barring calculators)

You are permitted to carry ANY relevant printed material to the test:

Notes, Textbooks, Reference Books, Linux Manual etc.

Combination of MCQs (Code review/Debugging) and T/F questions

Note:

- Both tests are open-book and assess the <u>application of knowledge at a deeper level</u> to solve systems programming problems.
- So, mere studying won't help. You need to consistently program with master's level competency to do well in the

Project

- You can work alone or in teams or two members
  - In case of teams, both students must be from the same section
- Project work will be evaluated through demo and viva
- In case of teams, project work will be evaluated individually (and separately) for each team member
- The system will not accept/permit submissions beyond the due-date and time.
- You need to include comments explaining the working of various modules of your solution.
- You are required to follow the submission instructions very carefully (reach out to the Professor if you have any questions)

Plagiarism

Your work will be compared with all other submissions (all four sections) manually and also with:

- MOSS (Measure of Software Similarity)-Stanford University

Do not share your work with anyone.

Sharing work is equally an academic offense!

- The GAs and the Instructor reserve the right to confirm the originality of code submissions through **one or more** of the following methods:
  - MOSS
  - Code Walkthroughs/Explanation
  - Debugging of simple bugs introduced in the submissions (code).

Submission Instructions

- All submissions have to be made over Brightspace within the submission deadline
- Only submissions in Brightspace that are submitted by the stipulated deadline will be marked
  - Email submissions to the GA or the instructor will not be permitted and will not be marked
- You are required to **precisely** follow the submission instructions
  - Ex1: uploading .docx file when .pdf file is required will not be acceptable
- 0 Marks will be awarded if submission instructions are not precisely followed

Marking Resolution

"Any marking-related concern has to be brought to the attention of the concerned GA within **5 calendar days** of the publishing of marks on Brightspace. If you are not satisfied with the resolution provided by the GA, you are required to contact the instructor within **10 calendar days** of the publishing of marks on Brightspace. Failure to do so will automatically result in no changes to the original marking / zero marks". ***//*As per the course outline**

The GAs and the Instructor reserve the right to confirm the originality of code submissions through <u>one or more</u> of the following methods:

- MOSS
- Code Walkthroughs/Explanation
- Debugging of simple bugs introduced in the submissions

Introduction to Unix –Outline

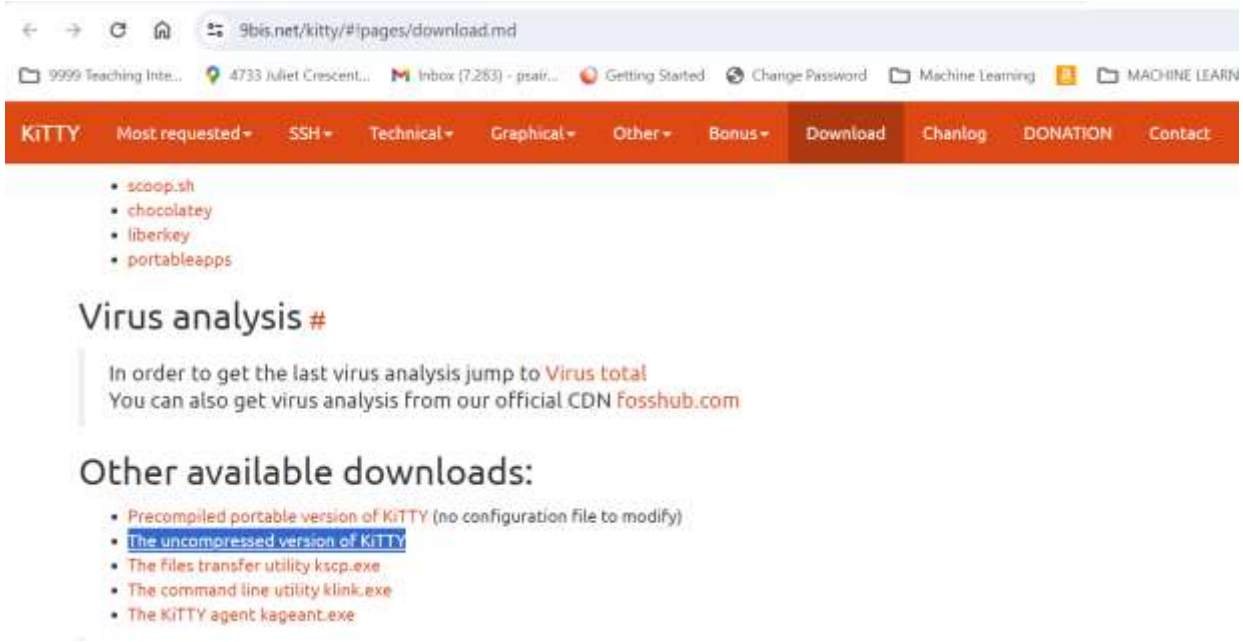
- Connecting to the CS Linux Server
- The Role of Operating Systems
- Architecture of the Unix Operating System
- GNU
- History of Unix
- Other implementations of UNIX
- Shells
- **File System**
- **Input Output**
- **Process and Process ID**
- **Signals**
- **Pipes**
- **System Calls and Library Functions**
- Conclusion and Summary

Connecting to the CS Linux Server

Install Global Protect (step 1)

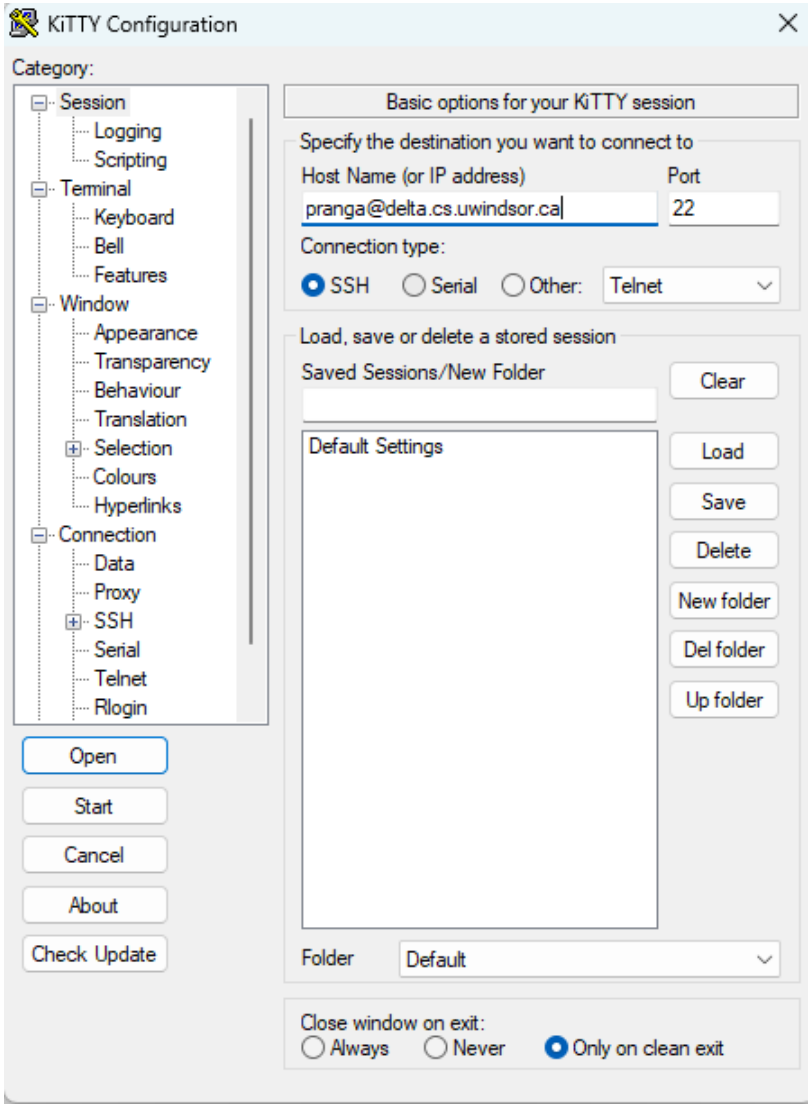
https://www.9bis.net/kitty/#!pages/download.md **(step 2)**

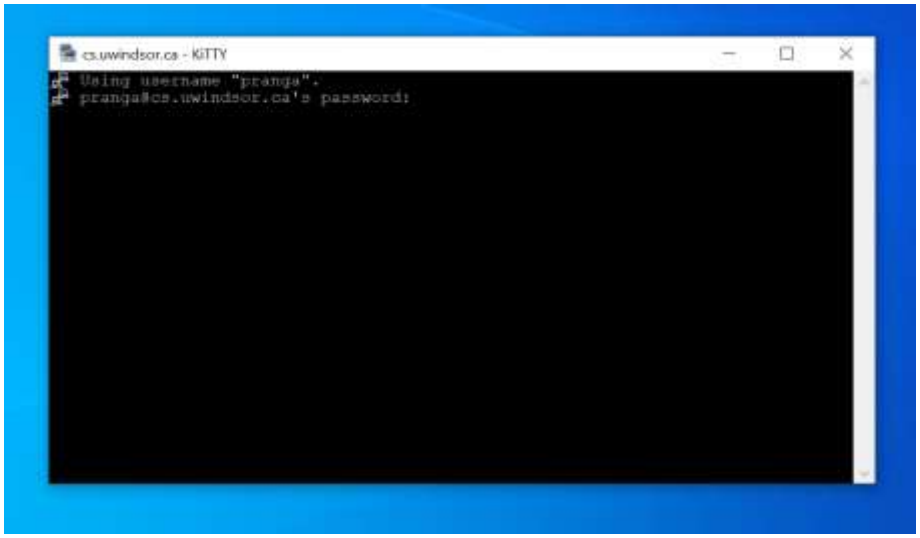
Download and RUN the uncompressed version of Ktty

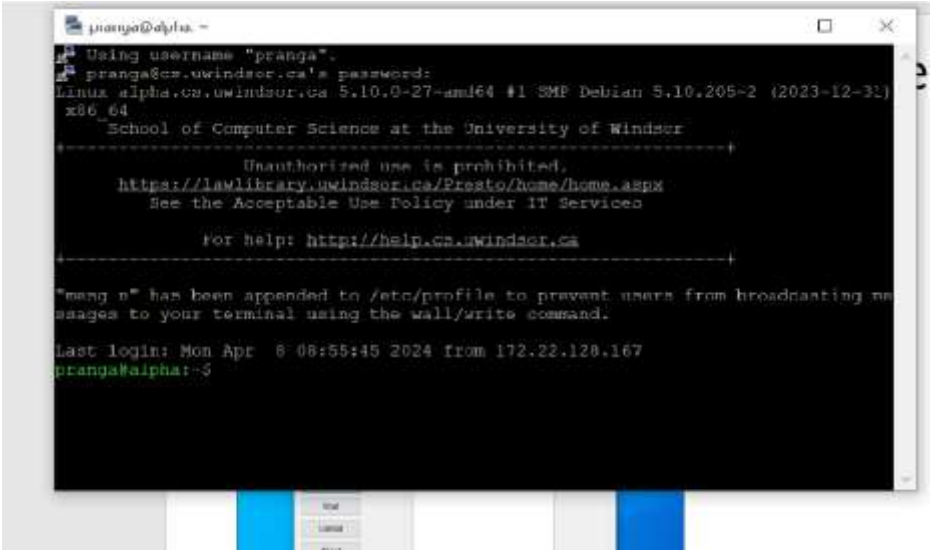


Connecting to the CS Linux Server..

Host Name (or IP address): <u>username@delta.cs.uwindsor.ca</u> //Port: 22







Kitty for MAC OS

https://www.macupdate.com/app/mac/62176/kitty

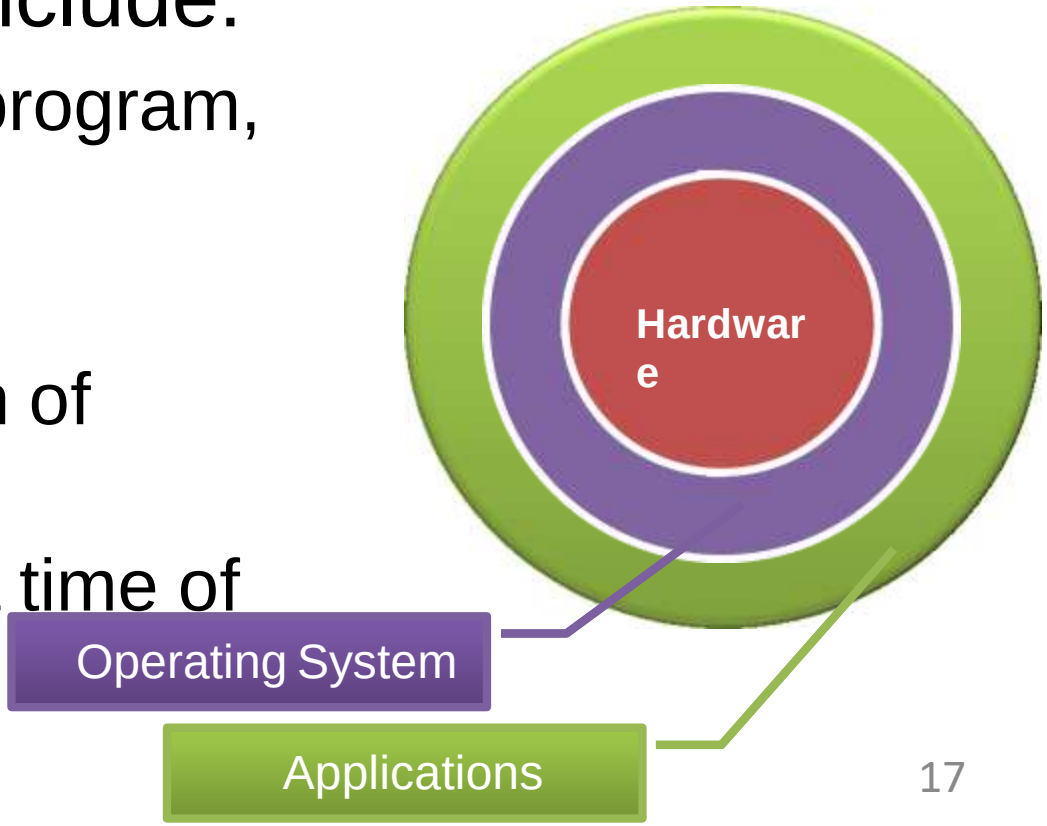
Sample C Program (welcome.c)

```
# include <stdio.h>  //welcome.c

main()
{

        printf("\nWelcome to COMP 8567\n");

}
```

```
$ gcc -o welcome welcome.c

$ ./welcome
```

Role of Operating Systems

- All operating systems provide services for programs they run.
- Typical <u>services</u> include:
  - executing a new program,
  - opening a file,
  - reading a file,
  - allocating a region of memory,
  - getting the current time of day,
  - and so on.

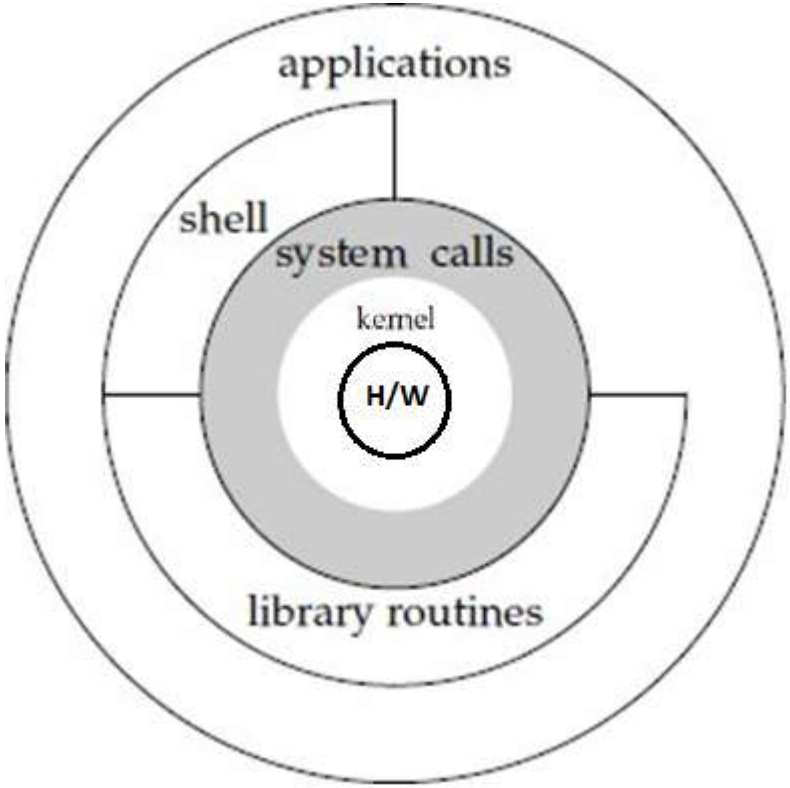


Major Software Components of OS

- Process Management
- Memory Management (Primary Memory)
- File Management (Secondary Memory)
- I/O Management
- Network Management
- Security Management

Architecture of the UNIX operating system

- The core of the UNIX OS is called the ***kernel.***
- The ***kernel*** directly interacts with the hardware and provides services to the applications
- The interface to the kernel is a layer of software called the ***system calls*** .
  - A system call is the programmatic way in which a computer program requests a service from the kernel of the operating system on which it is executed[1]
- **Libraries** of common functions are built on top of the system call interface, but applications are free to use both.
- The **shell** is a special application that provides an



For example, **Linux** is the <u>kernel</u> used by the GNU operating system.

Unix Essential Training
(LinkedIn Learning Course)
Due: Jan/14/2026, 11 pm EDT
Note: You will need a Windsor Public Library Card/Account to access LinkedIn Learning (Downtown Central Library would be the best place to get a card)

Unix Essential Training: (See the lab section in Brightspace)

Kevin Skoglund

https://www.linkedin.com/learning-login/share?account=2212217&forceAccount=false&redirect=https%3A%2F%2Fwww.linkedin.com%2Flearning%2Funix-essential-training%3Ftrk%3Dshare_ent_url%26shareId%3D4TPnT9DRRM2GS7x9xJWF2g%253D%253D

windsorpubliclibrary.com/databases/

9999 Teaching Inte... | Inbox (7,283) - psair... | Getting Started | Change Password | Machine Learning | MACHINE LEARNING | Gmail

- The Canadian Encyclopedia
- L'encyclopédie Canadienne
- Le grand dictionnaire terminologique

►ENTERTAINMENT

- CBC Corner / ► L'espace Radio-Canada
- Freegal Music – music, downloadable and streaming*
- CloudLibrary Streaming Video*
- Hoopla Digital -music, movies, television shows, downloadable & streaming*
- kanopy* – movies & documentaries

►LIFELONG LEARNING

- LearningExpressLibrary – dozens of practice tests for various certifications
- LinkedIn Learning* (library ID: windsorpl
- Transparent Language Online*
- Universal Class
- WPL Mobile*

►LOCAL HISTORY & GENEALOGY

- Ancestry Library (Remote access to this resource, temporarily provided courtesy of ProQuest and its partner Ancestry, is being discontinued as of December 31, 2021.)

History of Unix

- 1969-71 AT&T Bell Labs
- MULTICS: Multiplexed Information and Computing Service
  - Mainframe-TimeSharing- MULTICS was used to mange Time-sharing
- AT&T Stopped the project
- Side Project was developed by the employees who were interested
- MULTICS became a single user system UNICS (Uniplexed information and Computing Service)
  - UNICS was modified to support multiple users and eventually became Unix (x for multiple users)
- 1972: Unix was rewritten in C
  - Unix became portable to other hardware

History..

- 1975 –US govt bans AT&T from making any software
- AT& T gave away free software (Unix) to almost anyone who requested for it -Universities, Govt agencies and Corporations
- A number of mainframes were now running on Unix

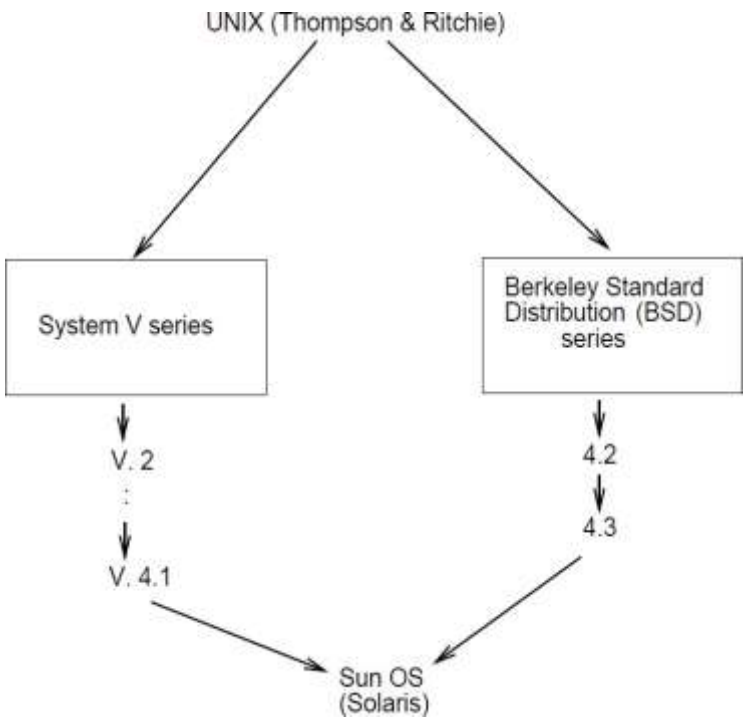
- First public version in 1975 is referred to as system 5 Open Source
  BSD, Linux

- Closed Source (Solaris, AIX etc) Proprietary HPUX

- Mixed Source (Mac OS / Darwin)
  IOS, ANDROID

Classic versions

- Two (classic) popular versions of Unix :
  - BSD (Berkeley Standard Distribution) Unix: introduction of Socket (Academics-Oriented)
  - System V: from Bell Laboratories (Industry-Oriented)



Other implementations of Unix

- **FreeBSD:** post BSD line after UCB decided to end work on BSD versions. Both binary and sources of the FreeBSD are freely available
- **Linux:** Freely available under GNU public license. Created in 1991, by **Linus Torvalds** to replace Minix
- **GNU** is a recursive acronym for "**GNU's Not Unix**," reflecting its goal of creating a system similar to Unix but entirely free from proprietary restrictions.
  - Became extremely popular(E.g., Debian, Ubuntu), in particular
    - **It is widely used on servers and mainframe computers**
    - **Android is built on top of the Linux kernel**

Where is Linux widely used?

- **Servers (Web/File/Database)**

Linux is used to power **96.3%** of the world's top 1 million web servers. Windows (1.9%), and FreeBSD 1.8%) are the other players:
https://www.enterpriseappstoday.com/stats/linux-statistics.html

. **Cloud Computing:** The majority of the cloud infrastructure around the world is run on Linux

- **Mobile Devices**
  - **Android** which is built on the Linux kernel dominates the mobile phone market
  - **iOS** is a variant of Darwin, derived from BSD, a UNIX-like kernel
- **macOS** is based on a BSD Unix kernel known as Darwin which is open-source.
- **Embedded Systems (**smart TVs, Industrial Automation, IoT, POS systems, Medical Imaging systems etc.) devices
- **Networking** (routers, switches, firewalls etc.)
- **Supercomputers** https://en.wikipedia.org/wiki/TOP500 (List of top 500 supercomputers)

Approximately what percentage of jobs in the IT industry require Linux skills?

Please Note

This is a graduate-level course in advanced systems programming and assumes students have a reasonable working knowledge of Linux

This course is not about teaching Linux commands!

Shells

- A ***shell*** is a command-line interpreter that reads user input and executes commands.
- The user input to a shell is normally from the **terminal** (an interactive shell) or sometimes from a file (called a ***shell script***).

Name	Path	FreeBSD 8.0	Linux 3.2.0	Mac OS X 10.6.8	Solaris 10
Bourne shell	`/bin/sh`	•	•	copy of `bash`	•
Bourne-again shell	`/bin/bash`	optional	•	•	•
C shell	`/bin/csh`	link to `tcsh`	optional	link to `tcsh`	•
Korn shell	`/bin/ksh`	optional	optional	•	•
TENEX C shell	`/bin/tcsh`	•	optional	•	•

Figure 1.2 Common shells used on UNIX systems

//Current Shell

$ echo $SHELL

//Various shells supported by the system

$ cd /

$ cd bin

$ ls *sh -1

//Get the Version of Linux

$ uname –a

FILE SYSTEM

File System

- The UNIX file system is a hierarchical arrangement of **directories and files.**
- Everything starts in the directory called ***root***, whose name is the single character /
- A ***directory*** is a file that contains directory entries. Logically, we can think of each directory entry as containing a filename along with a structure of information describing the attributes of files and other directories.
- The attributes of a file are such things as the type of file (regular file, directory), the size of the file, the owner of the file, permissions for the file (whether other users may access this file), and when the file was last modified.
- The `stat` function returns information containing all the attributes of a file.
- **$ stat sample.c**
- The home directory is denoted by the ~ symbol
- $cd , $cd ~

Filename

- The only two characters that **cannot** appear in a filename are the slash character (/) and the null character.
- Characters are **CASE-SENSITIVE**
- The slash separates the filenames that form a pathname and the null character terminates a pathname.
- For portability, POSIX.1**(Portable Operating System Interface-IEEE standard)** recommends restricting filenames to consist of the following characters: letters (a-z, A-Z), numbers (0-9), period (.), dash (-), and underscore (_).
- Two filenames are automatically created whenever a new directory is created: .(called *dot*) and .. (called *dot-dot*).
- **$ cd ..**
- **$ cd .**
- Dot refers to the current directory, and dot-dot refers to the parent directory. In the root directory, dot-dot is the same as dot.

Pathname

- A sequence of one or more filenames, separated by slashes and optionally starting with a slash, forms a *pathname*.
- A pathname that begins with a slash is called an ***absolute pathname***; otherwise, it's called a ***relative pathname***. Relative pathnames refer to files relative to the current directory.
- The name for the root of the file system (/) is a special-case absolute pathname that has no filename component.

Absolute and Relative Pathnames and Other file operations

$ cd Music

$ cd /home/pranga/Music

//Moving files from one folder to another
// cp, mv and rm

$ cp hello.c Music // Keeps the original file after copying
$ mv hello.c Music //Removes/deletes the original file after moving
$ cd Music
$ rm hello.c //Permanently deletes the original file
//Rename hello.c to helloworld.c
$ mv hello.c helloworld.c

Working Directory

- Every process has a *working directory*, sometimes called the *current working directory*.

  `pwd //`**`Print Working Directory`**

- This is the directory from which all relative pathnames are interpreted.

INPUT AND OUTPUT

Input and Output

- **Standard Input, Standard Output, and Standard Error**
- By convention, all shells open three descriptors whenever a new program is run: standard input, standard output, and standard error.
- If nothing special is done, as in the simple command
  `ls`
  then all three are connected to the terminal.
- Most shells provide a way to redirect any or all of these three descriptors to any file.
- For example, `ls > listing.txt` executes the ls command with its standard output redirected to the file named listing.txt
- (use < for input redirection, >> for concatenation of output)

PIPES

Pipes

- **Pipes** : a mechanism that allows the user to specify that the <u>output of one program</u> is to be used as the <u>input of another program</u>.
  - several programs can be connected in this fashion to make a pipeline of data owing from the first process thro

Example :

Ls -1 `| grep .c | wc -w`

PROCESSES AND PROCESS ID

Processes and Process ID

- An executing instance of a program is called a ***process***. (Some operating systems use the term *task* to refer to a program that is being executed).
- The UNIX System guarantees that every process has a <u>unique numeric identifier</u> called the ***process ID***. The process ID is always a non-negative integer.
- Try ps(Process Status)
- Every process has a **process id**, **parent process** id and **group id**.

Process ID, Parent ID, Group ID

```
#include <stdio.h>
#include <unistd.h>


// processids.c
//process id, parent id, process group id
        main(void)
        {


                printf("\n The current process id is %d \n",getpid());

                printf("\n The parent id of the current process is %d \n",getppid());

                printf("\n The group id of the current process is %d \n",getgid());

        }
```

Three Important Process- Related Operations

- fork() //Used to create a new child process
- exec() //Used to replace a child process with a new process
- waitpid() // Used to wait for a process to complete execution

SIGNALS

Signals

- Signals (Software Signals) are used to notify a process of the occurrence of some condition.
- For example, the following generate signals :
- A <u>division by zero</u> : the signal **SIGFPE** is sent to the responsible process that has three choices. Ignore the signal, terminate the process or, call a function to handle the situation.
- The <u>Control-C</u> key : when pressed, it generates a signal that causes the process receiving it to interrupt.
- Calling the function ***<u>kill</u>*** : a process can send a signal to another process causing its death. This is an example where Unix checks our permissions before allowing the signal to be sent.
  - A number of signals can be used along with the ***<u>kill</u>*** command: $ kill –l

Kill Processes – Example

Run Multiple Processes and Kill Processes

$ ps –u
$ kill - (signo) *Processid*

$Kill –l *//List of all signals*

SYSTEM CALLS AND LIBRARY FUNCTIONS

System calls and library functions

- "In computing, a system call is the programmatic way in which a computer program **requests a service from the kernel** of the operating system on which it is executed" [1]
- Each system call in Unix has an **interface function**, in the C standard library, with the same name that the user process invokes.
- The interface function then invokes the appropriate kernel service, using whatever technique is required on the system.
- An interface function for a system call cannot be re-written/overridden by the user
- For our purpose, a system call will be viewed as a **regular C function.**

System Calls- Common Examples

Open()
Read()
Write()
Close()
Fork()
Exec()
Exit()
Getpid()
Getgid()
Getuid()
--
--
--

A simple C program that prints the userid using the getuid() system call

```
// userid.c
//uses getuid() system call

#include <stdio.h>
#include <unistd.h>

	main(void)
	{
		printf("\nThe current userid is %d\n",getuid());
	}
```

System calls and library functions

- The functions which are a part of standard C library are known as Library functions: printf(), scanf(), strcmp(), strlen() etc are library functions.
- Note that a user process can invoke either a system call or a library function.
- A library function might invoke a system call.

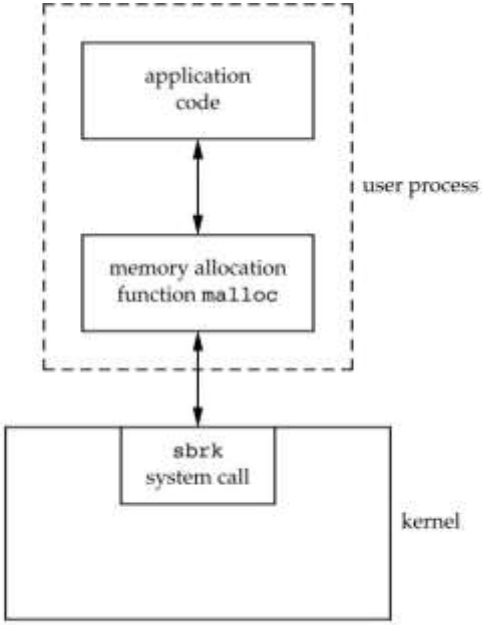


Figure 1.11 Separation of malloc function and sbrk system call

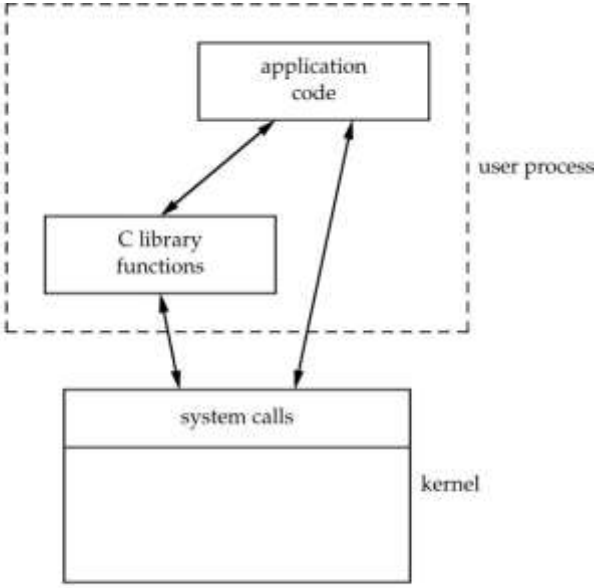


Figure 1.12 Difference between C library functions and system calls

Summary

- Operating system: Software to manage computer resources, in particular,
  - it runs a program for a user
  - it allows communication with devices and processes
- A program is a fille containing instructions
- A process is a program being executed
- Unix is a multiuser operating system
- Most of Unix is written in the C language
- Unix has a simple philosophy: a program should do one thing and do it well.
- Entry points in Unix are called system calls. They allow the user to get services from the kernel.

Reminder:

Unix Essential Training

(LinkedIn Learning Course)

Due: Jan/13/2025

Unix Essential Training: (See the lab section in Brightspace)

Kevin Skoglund

https://www.linkedin.com/learning-login/share?account=2212217&forceAccount=false&redirect=https%3A%2F%2Fwww.linkedin.com%2Flearning%2Funix-essential-training%3Ftrk%3Dshare_ent_url%26shareId%3DnHIsKhovToqnR9nuQigrsQ%253D%253D